Robust Fits

Exercise: Reconsider the smooth shown in the Figure on Slide 53 above. Does it appear to be a good fit?

It appears that extreme observations close to $X = 0$ are "pulling" the fitted model up, maybe more than we find acceptable

This is a result of using least squares

The problem in the fit is that the extreme values are overly influential. Least squares is particularly sensitive to these extremes; the large response values are "pulling up" the estimate.

Fortunately, `loess()` and `loess.as()` allow one to move away from least squares, by setting the argument `family="symmetric"` instead of the default `family="gaussian"`.

```
> holdlo3 = loess.as(optionsdata20$inmoney,
+         optionsdata20$implvol, family="symmetric",
+         criterion = "gcv")
```

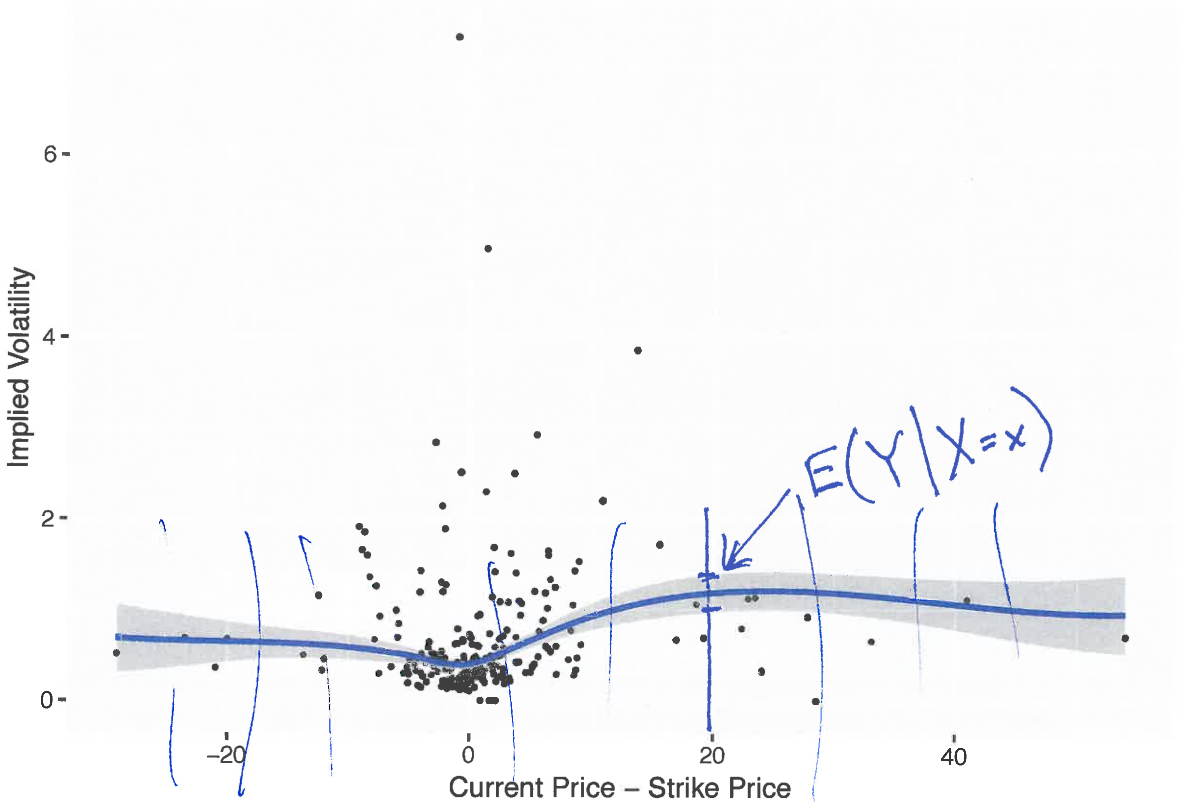
Exercise: What is the reasoning behind using the name `family`, with value `symmetric`, to specify a robust fit?

When assume errors are normal (family = "gaussian"), then least squares is optimal (the MLE)

When use family = "symmetric", assuming the error dist is symmetric, but has heavier tails than normal. Use penalty other than squared error to reduce influence of extremes

Now, we refit with this span, using `family="symmetric"`:

```
> ggplot(optionsdata20, aes(x=inmoney,y=implvol)) +
+   geom_point(size=0.5) +
+   geom_smooth(method="loess", span=holdlo3$pars$span,
+       method.args=list(degree=1,
+        family="symmetric")) +
+   labs(x="Current Price - Strike Price",
+        y="Implied Volatility")
```



Confidence Interval

The depicted gray region around the regression function estimate is a 95% pointwise confidence band for the regression function.

The **proper** interpretation is as follows: For each predictor value x, the gray region displays a 95% confidence interval for $f(x)$.

Here are two crucial things to understand regarding the band:

1. The shaded area does **not** show a **simultaneous** 95% confidence band for the **entire** regression function.
2. The shaded area does **not** show 95% prediction intervals for new observations.

Exercise: Discuss the two misconceptions given above, and how they differ from the real interpretation.

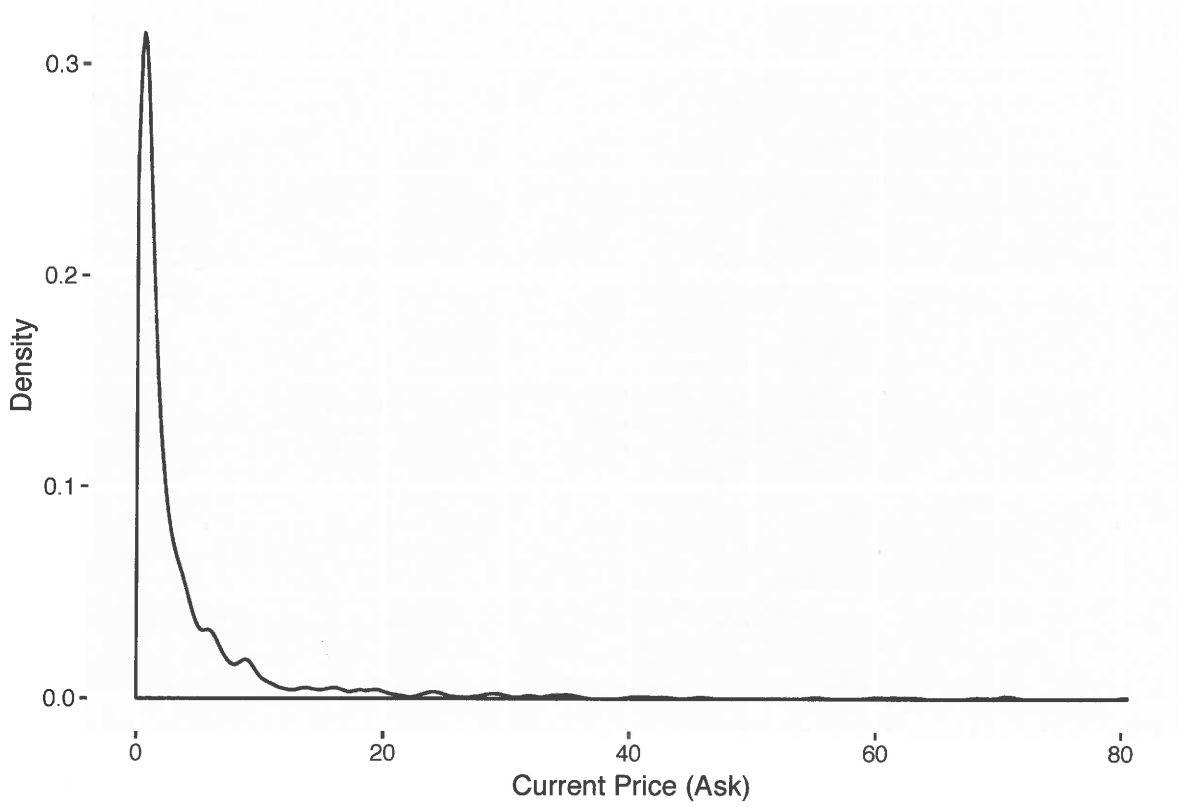
1. The pointwise band that we see could lead to "multiple testing problem." We are not 95% confident that all intervals cover true f(x).
2. If we created 95% prediction intervals from this band, we would greatly underestimate the errors in our predictions. The actual 95% prediction intervals would be much wider, and would cover most of the observed points.

Exploring Relationships

Now we will dig a little deeper into the options sample, and explore relationships between the relevant properties of an option and its current ask price.

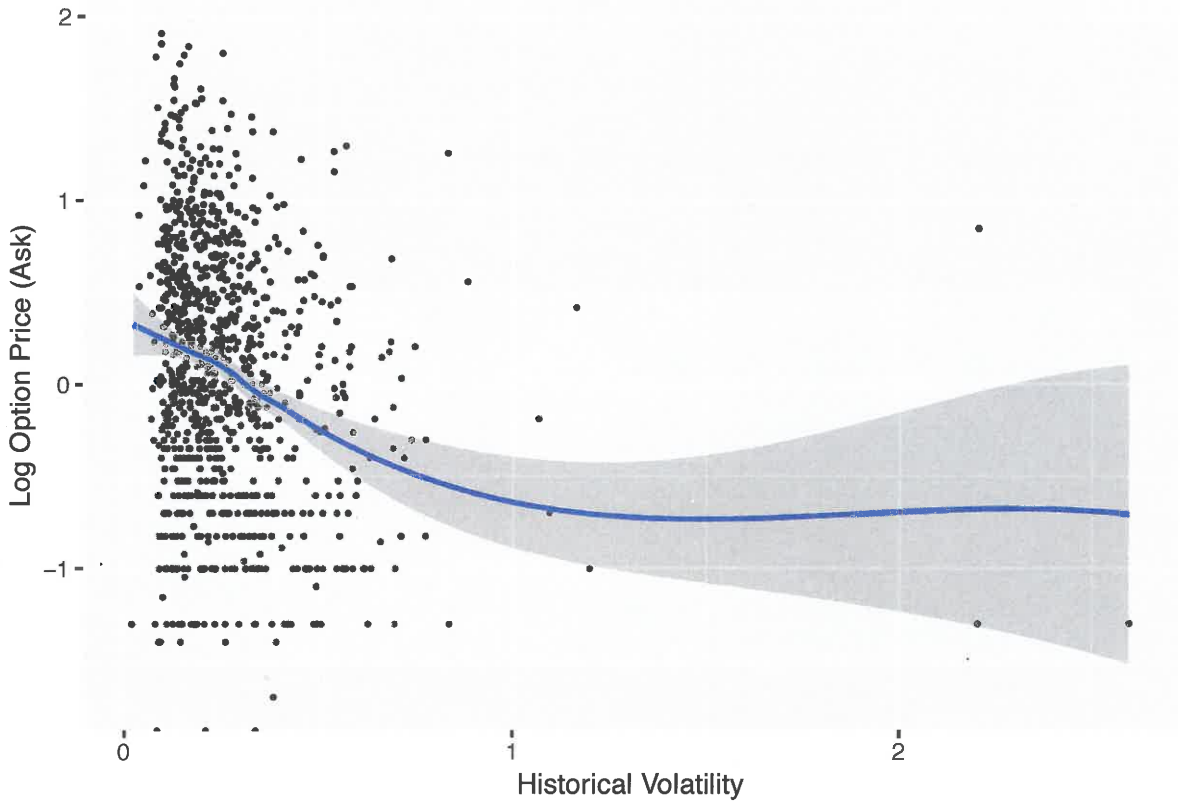
First, consider the distribution of the prices. There is a notable skew. In what follows we will work with the logarithm of the ask price.

```
> ggplot(optionsdata, aes(x=ask)) + geom_density() +
+   labs(x="Current Price (Ask)", y="Density")
```

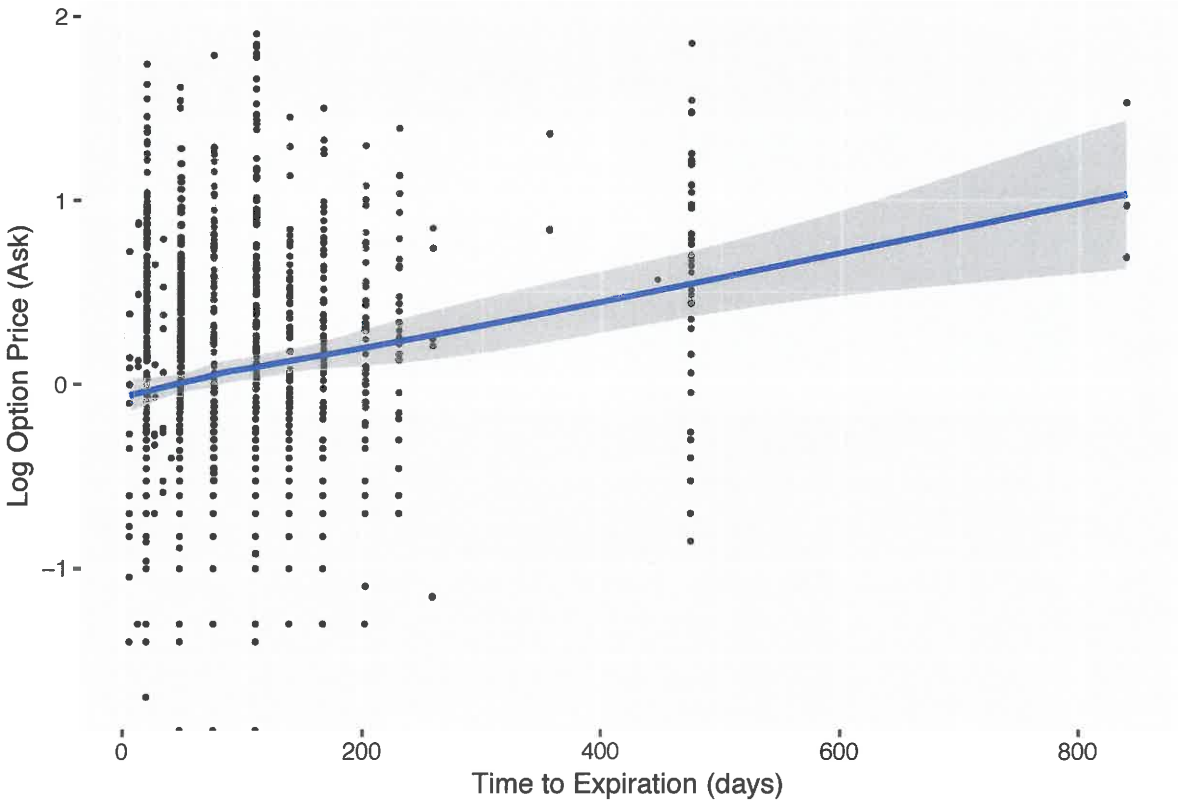


The next plots compare historical volatility, time to expiration, and strike price to the price. The code is only shown here for the first plot:

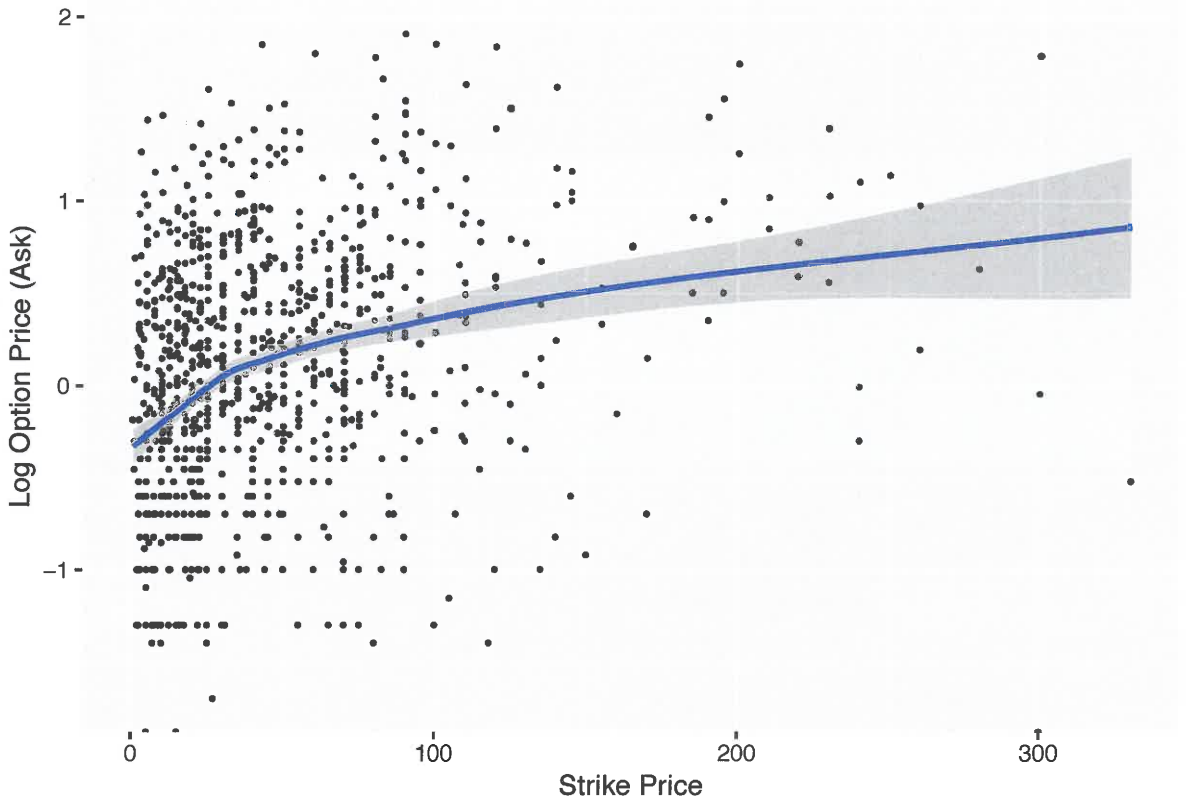
```
> ggplot(optionsdata, aes(x=histvol,y=log10(ask))) +
+   geom_point(size=0.5) +
+   geom_smooth(method="loess",
+               method.args=list(degree=1)) +
+   labs(x="Historical Volatility",
+        y="Log Option Price (Ask)")
```


Log Option Price (Ask)
2
1
0
−1
0
1
2
Historical Volatility

1


Log Option Price (Ask)
2
1
0
−1
0
200
400
600
800
Time to Expiration (days)

1



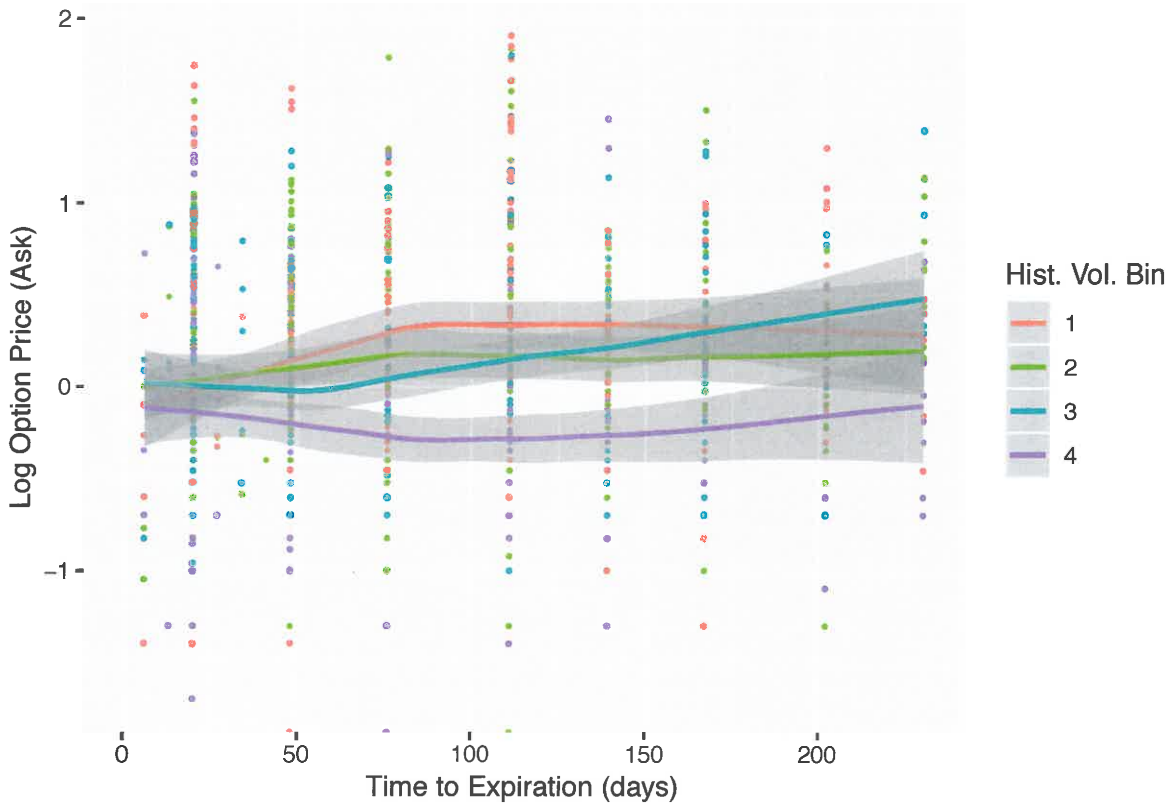
Exercise: One might guess that there is a strong relationship between the price of the option and how far "in the money" it currently is, i.e., the difference between the current price of the stock and its strike price. Construct a plot to explore this.

Next we will construct a plot attempting to explore how a pair of variables relate to the price.

Let's bin the historical volatility into four groups:

```
> breaks = c(-Inf, quantile(optionsdata$histvol,
+                           c(0.25,0.5,0.75)), Inf)
> optionsdata$histvolbin = cut(optionsdata$histvol,
+                         breaks, labels=c(1,2,3,4))

> ggplot(optionsdata, aes(x=timetoexpiry,y=log10(ask),
+                           color=histvolbin)) +
+   geom_point(size=0.5) +
+   geom_smooth(method="loess",
+               method.args=list(degree=1)) +
+   labs(x="Time to Expiration (days)",
+        y="Log Option Price (Ask)",
+        color="Hist. Vol. Bin") +
+   xlim(0,230)
```

Exercise: Interpret the graph on the previous page.

Part 8: Dimension Reduction

Summary

Often data come to us in a high-dimensional form, meaning that each observation is actually best thought of as a vector of many numbers.

In some situations, it is crucial that lower-dimensional representations of these data are obtained. Here we present some approaches to doing that.

First, we motivated why this dimension reduction can be so important.

The Curse of Dimensionality

Despite the promise of nonparametric approaches, they suffer from a serious drawback: The amount of data required to fit nonparametric density estimates or regressions well increases exponentially with the dimension of the data.

This is called the Curse of Dimensionality.

To start, we ask: **What is the "dimension" of data?**

Standard data come to us in the form of numbers, but a single observation is often best thought of as a list of numbers.

For example, a yield curve shows how bond rates vary as a function of time to maturity. A single yield curve consists of several numbers, usually around ten rates. This count of how many numbers it takes to represent a yield curve is called its dimension. We would say that a single yield curve is "ten-dimensional."

Another example: Imagine our data consists of separate price time series for each of 100 stocks. Each series goes back 20 trading days. We would say in this case that we have "100 observations of 20-dimensional time series".

One place where the dimension of data often comes up is in the context of regression or supervised learning. If there are p predictors being used in a model for a response variable, we would say that we are using a "p-dimensional predictor vector." $Y, \underset{\sim}{X} = (X_1, X_2, \ldots, X_p)$

Nonparametric regression is possible when the predictor vector is p-dimensional. For instance, if $p = 2$, a neighborhood can be thought of as a square centered on the target value. Local linear regression involves fitting a plane within this square.

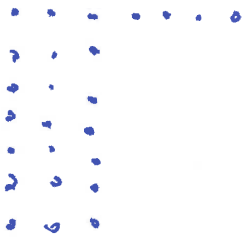
Reconsider our options sample from the previous Part. The figure on the following page shows a two-dimensional local linear regression fit. The response is the "error" in the Black-Scholes price (relative to the ask price) and the predictors are the time to expiration and the strike price.

First, read in the data and fit the model:

```
> optionsdata = read.table("optionssample09302017.txt",
+                          header=T, sep=",")
> optionsdata$bserr = optionsdata$bsval - optionsdata$ask
>
> holdlo2d = loess(bserr ~ timetoexpiry + strike,
+                  data=optionsdata, degree=1)
```

The function `expand.grid()` creates a two-dimensional grid of values from two vectors.

```
> fullgrid = expand.grid(
+   timetoexpiry=
+     seq(min(optionsdata$timetoexpiry),
+         max(optionsdata$timetoexpiry),length=20),
+   strike=seq(min(optionsdata$strike),
+              max(optionsdata$strike),length=20))
```



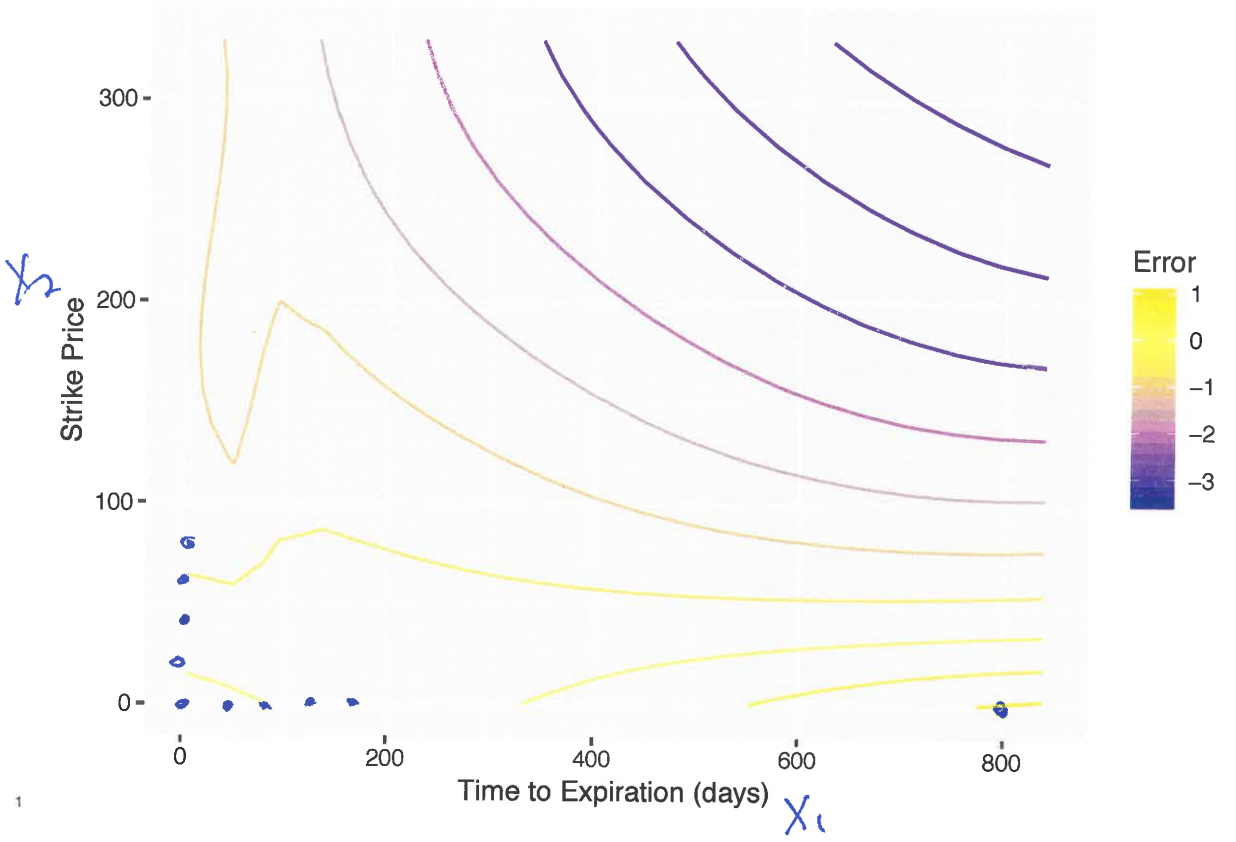
Now, make the predictions on this grid:

```
> fits = predict(holdlo2d,newdata=as.matrix(fullgrid))
```

Finally, create the plot. Note the use of the syntax `color=..level..` in order to specify that the colors should reflect the different levels in the contour plot.

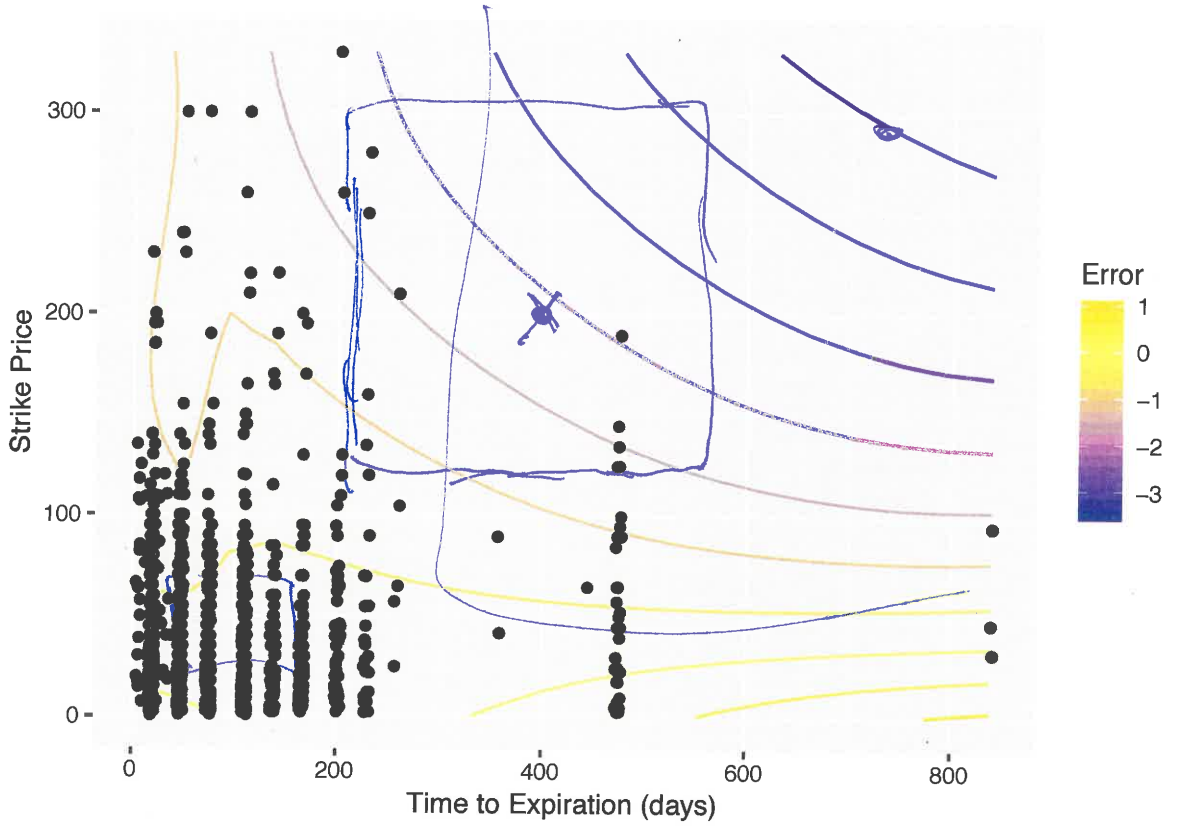
400 rows, 2 cols

```
> fitframe = data.frame(cbind(fullgrid,fits))
>
> ggplot(fitframe, aes(x=timetoexpiry,y=strike,z=fits)) +
+   geom_contour(aes(color=..level..)) +
+   scale_color_gradient2(low="blue", high="red",
+                         mid="yellow",midpoint=0) +
+   labs(x="Time to Expiration (days)", y="Strike Price",
+        color="Error")
```



Exercise: The next plot adds on the 1,000 observed predictor pairs used in this case. Sketch the neighborhood when the regression function is estimated at $(400, 200)$. What will have to happen in order for this estimate to have sufficiently low variance?

Need to choose the nbhd quite large, probably larger than I would like



This example shows the fundamental challenge of working with high-dimensional predictors: As the dimension increases, the available observations become more "spread out." Neighborhoods must be made larger in order to compensate and achieve estimates with acceptable variance.

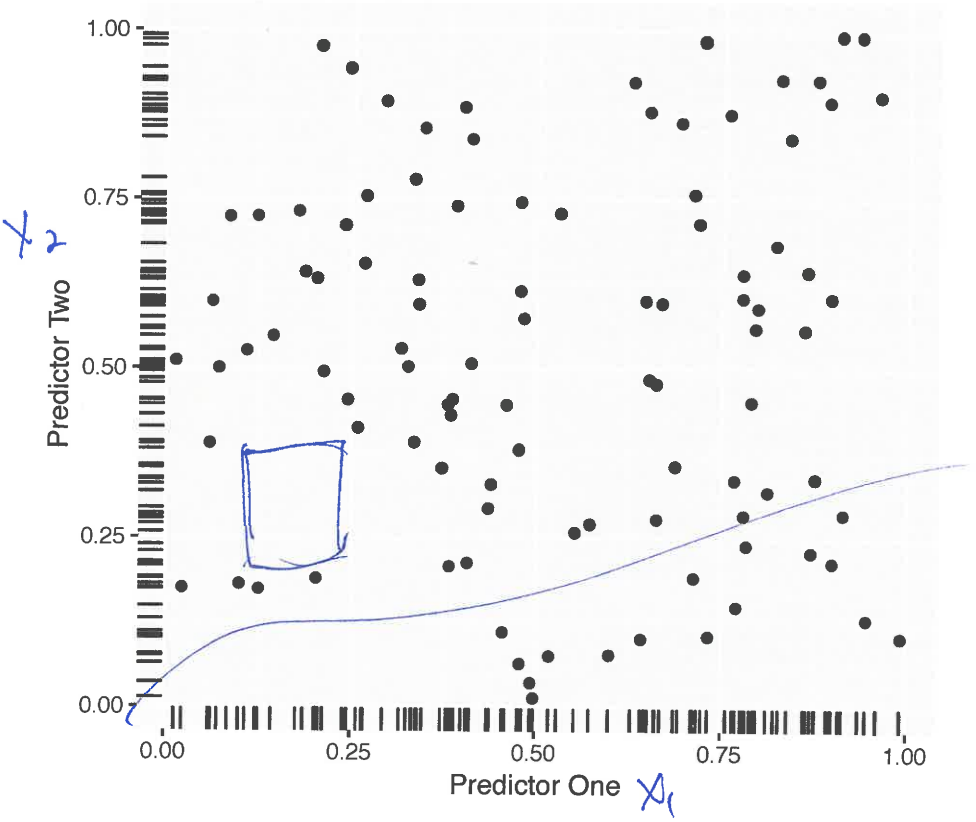
But, choosing a large neighborhood takes away a key feature of using non-parametric approaches: We want the estimate to be flexible, and not be based on too much "smoothing."

Consider the plot on the following slide.

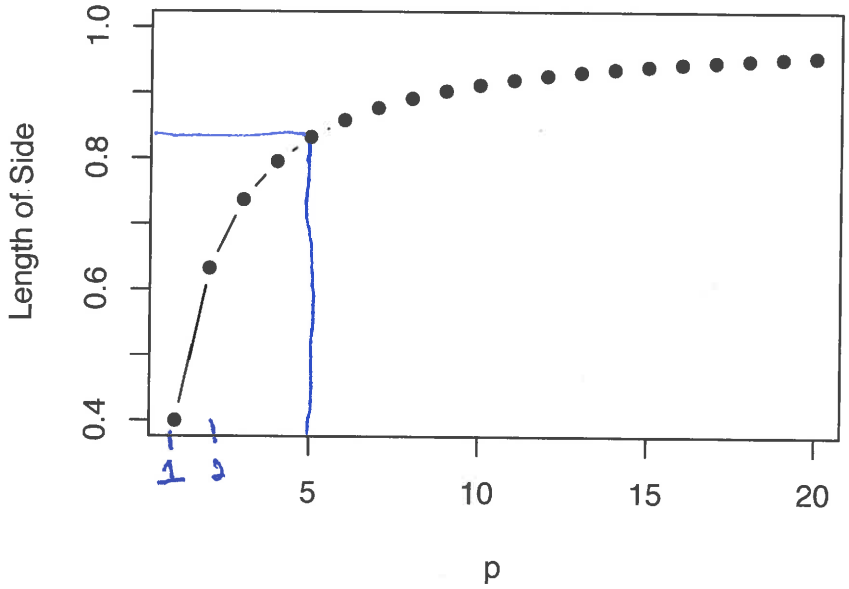
Here, the sample size is 100. Each observation is two-dimensional, i.e., imagine these are the two predictors. These data are simulated from uniform distributions.

The "marks" shown in each margin depict the marginal distribution for each predictor. Note that in the margins, the data appear to be quite closely spaced. If we were to fit a nonparametric regression using just one predictor, neighborhoods could be chosen small.

But, in two dimensions, large gaps appear in the distribution of the observations. Neighborhoods will have to be chosen larger in order to capture enough data.



Another way of thinking about it: Under the uniform setup, with a p-dimensional predictor, in order for a neighborhood to include proportion α of the data, the length of the sides of the neighborhood must be $\alpha^{1/p}$. This plot shows the case where $\alpha = 0.4$.



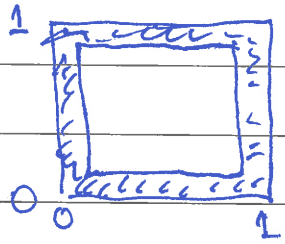
Exercise: Suppose that $p = 5$. Under our uniform setup, how long are the sides of a neighborhood that covers 40% of the data? What are the implications of this?

The sides have length $(0.4)^{1/5} \approx 0.82$. Recall that the predictor values only range from 0 to 1, so in each dimension my nbhd covers 82% of the range of values of the predictor.

Exercise: Again assume our uniform predictor distribution. Define that an observation is "close to the boundary" if it is either less than 0.1 or greater than 0.9 for at least one predictor. What proportion of the observations are "close to the boundary?"

When $p=1$, 20% of obs. are "close to boundary"

$p=2$, $1-(0.8)^2 = 0.36$



For general p, $1-(0.8)^p$

when $p=10$, $\approx .90$

The curse of dimensionality motivates us to consider methods which create low-dimensional representations of data.

In many cases, data which are p-dimensional in their original form can be compressed to q dimensions, where $q \ll p$, with minimal loss of important information.

The aforementioned yield curves provide an important example. The daily yield curve consists of approximately 10 rates, but a yield curve takes a limited class of shapes. Shifts in the yield curve can be described using three numbers in most situations. Hence, a 10-dimensional object can be described compressed to three dimensions.